Componentes e Troubleshooting

CronJobs, Secrets, RBAC e investigação de problemas



Ada · DE-OP-011 Orquestração · Kubernetes | Aula 07 | Everton J Benedicto

Cronograma das Aulas

01
17/07 · Sexta
Conhecendo o Kubernetes

02
20/07 · Segunda
Mão na Massa: Minikube e kubectl

03
22/07 · Quarta
Pods, Namespaces e Controladores

04
24/07 · Sexta
Primeira Aplicação no K8S

05
27/07 · Segunda
Estratégias de Deploy e Limitação de Recursos

06
29/07 · Quarta
Services, Ingress e Volumes

07
31/07 · Sexta
Componentes e Troubleshooting

08
03/08 · Segunda
Cluster Multi-Master

09
05/08 · Quarta
Concluir atividades do módulo e preencher rubricas

Ice Breaker

Como você faz para adquirir conhecimento?

(redes sociais, podcasts, etc...)

Objetivos de hoje

- Agendar tarefas com CronJobs
- Proteger dados sensíveis com Secrets
- Controlar acessos com RBAC
- Dominar as ferramentas de troubleshooting
- Investigar problemas de Pods, Services e do Cluster

1. Componentes finais

CronJobs, Secrets e RBAC

CronJob: para que serve e como funciona

Automatiza tarefas repetitivas que rodam em horários ou intervalos definidos — backups, envio de e-mails, limpeza de dados, funcionando como o cron do Linux.

`apiVersion: batch/v1 · kind: CronJob` — declara ao Kubernetes um recurso do tipo CronJob.

`metadata.name: hello` — nome que identifica o CronJob dentro do cluster.

`schedule: "* * * * *"` — periodicidade no padrão Cron (minuto, hora, dia do mês, mês, dia da semana); cinco asteriscos = executar a cada minuto.

`jobTemplate` — modelo do Job disparado a cada horário agendado.

`containers` — o que roda dentro do container temporário.

`image: busybox` — imagem Linux mínima e leve para executar a tarefa.

`command` — exibe a data/hora atual (date) e imprime "Hello from the Kubernetes cluster".

`restartPolicy: OnFailure` — se o container falhar, o Kubernetes o executa novamente até obter sucesso.

CronJob: tarefas agendadas

cronjob.yaml – executa periodicamente, como o cron do Linux

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"   # a cada minuto
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: hello
            image: busybox
            command:
            - /bin/sh
            - -c
            - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

Secret: dados sensíveis

secret.yaml – como o ConfigMap, mas para senhas, tokens e chaves

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: secret-basic-auth
type: kubernetes.io/basic-auth
stringData:
  username: admin
  password: t0p-Secret
```

Secret: dados sensíveis

pod-secret.yaml – adicionando as secrets ao pod

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: secret-basic-auth
```

Secrets

MÃO NA MASSA

Crie a Secret:

```
$ kubectl create -f secret.yaml
```

Vejo o yaml da secret criada:

```
$ kubectl get secret -o yaml
```

Criando um Pod e associando a secret:

```
$ kubectl apply -f pod-secret.yaml
```

Secret: dados sensíveis

```
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-07> kubectl create -f secret.yaml
secret/secret-basic-auth created
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-07> kubectl get secret -o yaml
apiVersion: v1
items:
- apiVersion: v1
  data:
    password: dDBwLVNlY3JldA==
    username: YWRtaW4=
  kind: Secret
  metadata:
    creationTimestamp: "2026-07-30T22:45:40Z"
    name: secret-basic-auth
    namespace: default
    resourceVersion: "588"
    uid: 103295f4-1d86-4f19-85c8-8329d301b08e
  type: kubernetes.io/basic-auth
kind: List
metadata:
  resourceVersion: ""
PS C:\UserData\        \Documents\Private\mine\ada\orquestracao-k8s\aula-07> kubectl apply -f pod-secret.yaml
pod/mypod created
```

Fonte da imagem: Ada Tech

RBAC: controle de acesso por função

- RBAC = Role-Based Access Control (controle baseado em funções)
- Role — permissões dentro de um namespace
- ClusterRole — permissões válidas em TODO o cluster
- Só existem regras de permissão (verbs: get, list, create, delete...)
- RoleBinding / ClusterRoleBinding — associam a Role a usuários/grupos

Role

Nessa role, está sendo permitido que os usuários possam visualizar as informações relacionadas ao recurso do tipo Pods.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

ClusterRole

Perceba que a diferença entre elas é que o **Namespace deixa de existir na ClusterRole** e que ambas não possuem usuários ou grupos informados. Esse papel, de definir para quem essas regras serão válidas, é do *RoleBinding* and *ClusterRoleBinding*

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  # "namespace" foi removido, pois clusterroles são válidas em todo o cluster
  name: secret-reader
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "watch", "list"]
```

RoleBinding

Associe a Role a um ou mais usuários ou grupos.

```
apiVersion: rbac.authorization.k8s.io/v1
# Essa role permite a "jane" ler os Pods no Namespace "default"
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
# podemos especificar mais de um usuário ou grupo
- kind: User
  name: jane # o campo "name" é case sensitive
  apiGroup: rbac.authorization.k8s.io
roleRef:
  # "roleRef" especifica a qual Role/ClusterRole estamos associando essa RoleBinding
  kind: Role # os valores válidos nesse campo são Role or ClusterRole
  name: pod-reader # o nome precisa ser igual ao da Role
  apiGroup: rbac.authorization.k8s.io
```

ClusterRoleBinding

Associe a ClusterRole a um ou mais usuários ou grupos.

```
apiVersion: rbac.authorization.k8s.io/v1
# Essa role permite ao grupo "manager" ler "secrets" em qualquer namespace
kind: ClusterRoleBinding
metadata:
  name: read-secrets-global
subjects:
# podemos especificar mais de um usuário ou grupo
- kind: Group
  name: manager # o campo "name" é case sensitive
  apiGroup: rbac.authorization.k8s.io
roleRef:
  # "roleRef" especifica a qual Role/ClusterRole estamos associando essa RoleBinding
  kind: ClusterRole # os valores válidos nesse campo são Role or ClusterRole
  name: secret-reader # o nome precisa ser igual ao da ClusterRole
  apiGroup: rbac.authorization.k8s.io
```

2. Troubleshooting

Investigando quando algo dá errado

A caixa de ferramentas do diagnóstico

- describe — detalhes e eventos de um recurso
- get events — linha do tempo de eventos do cluster
- logs — saída da aplicação dentro do container
- exec — entrar no container para investigar
- Troubleshooting = achar a causa raiz e resolver

describe, events e logs

MÃO NA MASSA

Crie o namespace caso não o tenha

```
$ kubectl create ns aplicacao
```

Crie um pod

```
$ kubectl run --image=nginx nginx -n aplicacao
```

Detalhes e eventos de um Pod (primeiro passo sempre):

```
$ kubectl describe pods nginx -n aplicacao
```

Eventos do namespace inteiro:

```
$ kubectl get events -n aplicacao
```

Filtrar só avisos/erros:

```
$ kubectl get events --field-selector type=Warning
```

Logs da aplicação:

```
$ kubectl logs nginx -n aplicacao
```

Logs anteriores a um restart:

```
$ kubectl logs nginx --previous
```

describe, events e logs

```
PS C:\UserD                cuments\Private\mine\ada\orquestracao-k8s> kubectl create ns aplicacao
namespace/aplicacao created
PS C:\UserData\z004stze\Documents\Private\mine\ada\orquestracao-k8s> kubectl run --image=nginx nginx -n aplicacao
pod/nginx created
PS C:\UserData\                nts\Private\mine\ada\orquestracao-k8s> kubectl describe pods nginx -n aplicacao
Name:             nginx
Namespace:        aplicacao
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Fri, 31 Jul 2026 18:41:50 +0100
Labels:           run=nginx
Annotations:      <none>
Status:           Running
IP:               10.244.0.3
IPs:
  IP:  10.244.0.3
Containers:
  nginx:
    Container ID:   docker://b24fa5ad99e6769ee333beb11c9520eabbf9f840775a040437c010171e9e26a3
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Fri, 31 Jul 2026 18:42:27 +0100
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-cgqzj (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-cgqzj:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
```

Fonte da imagem: Ada Tech

describe, events e logs

```
PS C:\UserData                nts\Private\mine\ada\orquestracao-k8s> kubectl get events -n aplicacao
LAST SEEN   TYPE     REASON      OBJECT      MESSAGE
110s        Normal   Scheduled   pod/nginx   Successfully assigned aplicacao/nginx to minikube
109s        Normal   Pulling     pod/nginx   Pulling image "nginx"
73s         Normal   Pulled      pod/nginx   Successfully pulled image "nginx" in 36.113s (36.113s including waiting). Image size: 161327548 bytes.
73s         Normal   Created     pod/nginx   Container created
73s         Normal   Started     pod/nginx   Container started
PS C:\UserData\               ts\Private\mine\ada\orquestracao-k8s> kubectl get events --field-selector type=Warning
No resources found in default namespace.
PS C:\UserData\               ts\Private\mine\ada\orquestracao-k8s> kubectl get events --field-selector type=Warning -n aplicacao
No resources found in aplicacao namespace.
PS C:\UserData\               ts\Private\mine\ada\orquestracao-k8s> kubectl logs nginx -n aplicacao
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/31 17:42:27 [notice] 1#1: using the "epoll" event method
2026/07/31 17:42:27 [notice] 1#1: nginx/1.31.3
2026/07/31 17:42:27 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/07/31 17:42:27 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/07/31 17:42:27 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/07/31 17:42:27 [notice] 1#1: start worker processes
2026/07/31 17:42:27 [notice] 1#1: start worker process 29
2026/07/31 17:42:27 [notice] 1#1: start worker process 30
2026/07/31 17:42:27 [notice] 1#1: start worker process 31
2026/07/31 17:42:27 [notice] 1#1: start worker process 32
2026/07/31 17:42:27 [notice] 1#1: start worker process 33
2026/07/31 17:42:27 [notice] 1#1: start worker process 34
2026/07/31 17:42:27 [notice] 1#1: start worker process 35
2026/07/31 17:42:27 [notice] 1#1: start worker process 36
2026/07/31 17:42:27 [notice] 1#1: start worker process 37
2026/07/31 17:42:27 [notice] 1#1: start worker process 38
2026/07/31 17:42:27 [notice] 1#1: start worker process 39
2026/07/31 17:42:27 [notice] 1#1: start worker process 40
2026/07/31 17:42:27 [notice] 1#1: start worker process 41
2026/07/31 17:42:27 [notice] 1#1: start worker process 42
2026/07/31 17:42:27 [notice] 1#1: start worker process 43
2026/07/31 17:42:27 [notice] 1#1: start worker process 44
PS C:\UserData\               ts\Private\mine\ada\orquestracao-k8s> kubectl logs nginx --previous
error: error from server (NotFound): pods "nginx" not found in namespace "default"
PS C:\UserData                nts\Private\mine\ada\orquestracao-k8s> kubectl logs nginx --previous -n aplicacao
Error from server (BadRequest): previous terminated container "nginx" in pod "nginx" not found
```

Fonte da imagem: Ada Tech

exec: entrando no container

MÃO NA MASSA

Abrir um shell interativo dentro do container:

```
$ kubectl exec -it nginx -n aplicacao -- /bin/sh
```

Rodar um comando único sem abrir shell:

```
$ kubectl exec nginx -n aplicacao -- ls /app
```

Ler um arquivo de dentro do container:

```
$ kubectl exec nginx -n aplicacao -- cat /var/log/nginx/access.log
```

Status de erro mais comuns em Pods

- Pending — não coube em nenhum node (falta recurso ou taint)
- ImagePullBackOff — não conseguiu baixar a imagem (nome/tag errados)
- CrashLoopBackOff — a app quebra ao subir, repetidamente
- Terminating (preso) — deleção travada; force com --grace-period=0 --force
- Sempre investigue com describe + logs antes de agir

Status de erro mais comuns em Pods

```
Events:
  Type     Reason   Age                    From     Message
  ----     ------   ----                   ----     -------
  Normal   Pulling  26m (x141 over 15h)    kubelet  Pulling image "brookybru/kubernetes:hello-python"
  Warning  Failed   6m19s (x126 over 14h)  kubelet  Failed to pull image "brookybru/kubernetes:hello-python": rpc error: code = Unknown desc = Error response from daemon: Get "https://registry-1.docker.io/v2/":
net/http: request canceled while waiting for connection (Client.Timeout exceeded while awaiting headers)
  Normal   BackOff  93s (x3166 over 14h)   kubelet  Back-off pulling image "brookybru/kubernetes:hello-python"
```

Fonte da imagem: Ada Tech

Troubleshooting de Cluster e Nodes

- kubectl get nodes — algum node NotReady?
- kubectl describe node <nome> — condições e IP interno
- Logs dos componentes no node: kube-apiserver, scheduler, kubelet...
- ETCD: é o banco do cluster — falhas exigem backup e restore
- kubectl cluster-info dump — despejo completo para análise

Troubleshooting de Cluster e Nodes

```
[bruna.souza@localhost ~]$ kubectl  get svc nginx -n aplicacao
NAME    TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort    10.98.119.56   <none>        80:31756/TCP   22h
```

Fonte: Let's Code

Recapitulando a Aula 07

- ✓ CronJob agenda tarefas; Secret protege dados sensíveis
- ✓ RBAC controla quem pode o quê (Role + RoleBinding)
- ✓ Diagnóstico: describe → logs → exec; get events para o macro
- ✓ Conheça os status de erro e sua investigação específica
- ✓ Próxima aula: cluster Multi-Master do zero!

